# Internship Task 1: Web service and database in Docker

## Objective

Rework the **Test Task 2** solution so that the web service and the database run in **separate Docker containers** and are started together with **Docker Compose**.

## Architecture

Two containers managed by `docker-compose.yml`:

- `web` — the **Apache** web server serving the Flask video web service via **mod_wsgi**.
- `db` — **PostgreSQL** (replaces SQLite from Test Task 2).

## Constraints

- Docker + Docker Compose.
- Web server: **Apache** with **mod_wsgi** (Python WSGI).
- Database: **PostgreSQL** (driver `psycopg2`).
- Data and uploaded videos must survive container restarts (volumes).
- DB connection settings must be passed via environment variables (no hard-coding).
- The web service must wait for the database to be ready before accepting requests.

## Deliverables

1. `Dockerfile` — builds the Apache + Flask image.
2. `docker-compose.yml` — services `web` and `db`, volumes, healthcheck.
3. `web_service.py` — migrated from SQLite to PostgreSQL.
4. `.wsgi` file and Apache configuration (VirtualHost + WSGI).
5. `requirements.txt` — updated (`psycopg2`, and `mod-wsgi` if used).
6. `README` — run instructions (`docker compose up --build`).

# 1. Services

- `web` maps host port `8000` to Apache's port `80`.
- `db` uses the official `postgres:16` image.
- Named volume `db_data` for PostgreSQL data (`/var/lib/postgresql/data`).
- Named volume `uploads` for the `uploads/` folder.
- `web` must depend on `db` with `condition: service_healthy`.

# 2. docker-compose.yml structure (reference)

```
services:
  web:
    build: .                  # built from the Dockerfile (Apache + Flask)
    ports:
      - "8000:80"             # host 8000 -> Apache 80 inside the container
    volumes:
      - uploads:/app/uploads
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: videos
      DB_USER: video_user
      DB_PASSWORD: video_pass   # in production use .env / Docker secrets
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: videos
      POSTGRES_USER: video_user
      POSTGRES_PASSWORD: video_pass
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U video_user -d videos"]
      interval: 5s
      timeout: 5s
      retries: 10

volumes:
  db_data:
  uploads:
```

# 3. SQLite → PostgreSQL migration

- Connection: `sqlite3.connect(...)` → `psycopg2.connect(host, port, dbname, user, password)`.
- Placeholders: `?` → `%s`.
- Auto-increment: `INTEGER PRIMARY KEY AUTOINCREMENT` → `SERIAL PRIMARY KEY` (or `GENERATED ALWAYS AS IDENTITY`).

- Time: `datetime('now')` → `NOW()`.
- Upsert stays similar: `ON CONFLICT (company_name) DO UPDATE ...` (requires `UNIQUE` on `company_name`).
- Connections: open a new connection per request (psycopg2 connections are not thread-safe) or use a connection pool.

# Acceptance checklist

- [ ] `docker compose up --build` starts both containers without errors.
- [ ] `POST /upload` uploads a video and returns a working URL.
- [ ] `GET /get_video_url?company_name=...` returns the URL.
- [ ] After `docker compose down` and `docker compose up`, data and videos are preserved.
- [ ] `web` waits for `db` readiness and does not crash on startup.

# Hints / pitfalls

- For Apache + Python use `mod_wsgi`. The easiest way to build it for your Python is: `pip install mod-wsgi`, then `mod_wsgi-express module-config` prints the `LoadModule` line for the Apache config. The system package `libapache2-mod-wsgi-py3` must match the Python version in the image.
- A `.wsgi` file usually looks like: `from web_service import app as application`.
- Serving `uploads/` can stay on Flask (`send_from_directory`), but in production it's better to expose it through Apache with `Alias`.
- Do not commit secrets (`DB_PASSWORD`) to git — use `.env` / Docker secrets.
- Inspect inside a container: `docker compose exec web curl http://localhost/`.